

# DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs

Amir Fakhim Babaei

Department of Electrical & Computer Engineering,  
Virginia Tech  
Arlington, Virginia, USA  
babaei@vt.edu

Thidapat Chantem

Department of Electrical & Computer Engineering,  
Virginia Tech  
Arlington, Virginia, USA  
tchantem@vt.edu

**Abstract**—The widespread use of Deep Neural Networks (DNNs) is limited by high computational demands, especially in constrained environments. GPUs, though effective accelerators, often face underutilization and rely on coarse-grained scheduling. This paper introduces DARIS, a priority-based real-time DNN scheduler for GPUs, utilizing NVIDIA’s MPS and CUDA streaming for spatial sharing, and a synchronization-based staging method for temporal partitioning. In particular, DARIS improves GPU utilization and uniquely analyzes GPU concurrency by oversubscribing computing resources. It also supports zero-delay DNN migration between GPU partitions. Experiments show DARIS improves throughput by 15% and 11.5% over batching and state-of-the-art schedulers, respectively, even without batching. All high-priority tasks meet deadlines, with low-priority tasks having under 2% deadline miss rate. High-priority response times are 33% better than those of low-priority tasks.

## I. INTRODUCTION

Deep Neural Networks (DNNs) demand high computational power and challenge resource-constrained systems. GPUs improve DNN training and inference [1], but sequential processing leads to underutilization, requiring multiple parallel (multi-tenant) DNN scheduling [2]. Inference tasks in constrained settings often need real-time performance [3], common in fields like autonomous driving [4], healthcare [5], [6], AI at the edge [7], and NLP [8], [9]. GPU inference servers batch inputs for utilization [10], [11], but real-time schedulers cannot typically use input batching, as they require immediate task handling [12].

NVIDIA provides various concurrency mechanisms for designers, but no general guideline exists on the efficient GPU configurations and resource partitioning strategies in the literature. Most works isolate resources, with some exploring oversubscription [13]. Also, most works focus on maximizing throughput rather than achieving predictable real-time performance. GPUs have a gray-box architecture that makes timing predictions difficult. Approaches that consider *Worst-Case Execution Time* (WCET) prediction often either sacrifice throughput [14] or result in overly pessimistic estimates [15].

Each GPU has  $N_{SM,max}$  *Streaming Multiprocessors* (SMs), its smallest independent units. GPU processes, or *kernels*, may number in the hundreds per DNN. Kernels run sequentially in *CUDA Streams* (referred to as streams after this), and running multiple streams in parallel improves GPU utilization by reducing SMs’ idle time. With Multi-Process Service (MPS), we can create multiple *CUDA Contexts* (referred to as contexts after this), each assigned a portion of SMs. When total SMs allocated to contexts surpass  $N_{SM,max}$ , it is termed *Oversubscription*.

In this paper, we propose *DARIS*, a Deadline-Aware Real-Time DNN Inference Scheduler to address efficient GPU concurrency configuration challenges. DARIS can provide better predictability for periodic soft real-time tasks with two priority levels. Our primary goals are to minimize the deadline miss rate and maximize overall throughput. While we cannot guarantee that all high-priority

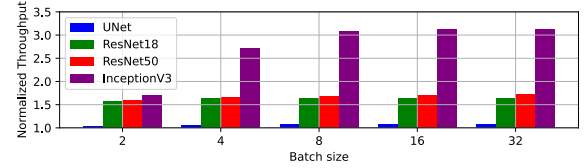


Fig. 1: Normalized throughput using batching

tasks will meet their deadlines, DARIS demonstrates significantly fewer deadline misses compared to state-of-the-art methods. We also introduce *staging* as a coarse-grained preemption mechanism to achieve better priority-based scheduling. Our main contributions are as follows:

- To the best of our knowledge, this is the first work to conduct an in-depth analysis of GPU concurrency mechanisms, focusing on resource oversubscription and its benefits on real platforms.
- We propose a deadline-aware real-time spatio-temporal GPU scheduler that surpasses the throughput of single-tenant batching, without relying on batched inputs.

## II. DESIGN CHOICES

### A. What Concurrency Mechanism to Use?

SGPRS [16] defines “knee point” as the maximum number of DNNs a scheduler can handle without missing deadlines. They found that without temporal partitioning, throughput drops significantly beyond the knee point due to interference. Higher oversubscription values generally improve throughput, though not consistently. These results suggest that a spatio-temporal scheduler is more effective than a purely spatial one, but further analysis is needed to comprehensively examine contexts, streams, and oversubscription levels.

### B. Is Oversubscription Good?

Systematic studies on SM oversubscription are limited. NVIDIA suggests 200% SM oversubscription as optimal [17], though some research indicates it may cause interference [18]–[20], with limited experimental data. Laius [13] uses structured oversubscription, dedicating 100% of SMs to user-facing and another 100% to batched services. SGPRS [16] highlighted the benefits of oversubscription, with the remaining challenge of finding the best trade-off. In **Section VI-E**, we compare oversubscription levels, denoted  $OS \geq 1$ , to find this balance.

### C. Is Batching Enough or Necessary?

Batching boosts throughput in GPUs but is often impractical for real-time inference with non-identical DNNs since waiting for jobs can cause missed deadlines. This study examines whether batching

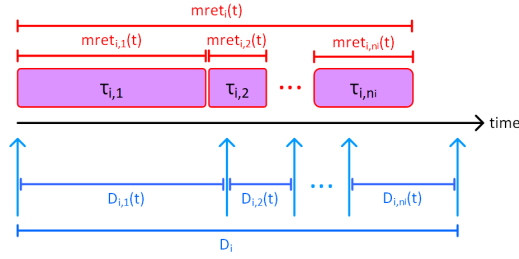


Fig. 2: Task staging and virtual deadline assignment

is sufficient and if its throughput can be matched without it. Experiments (Figure 1) show limited benefits for some DNNs (e.g., UNet) and significant gains for others (e.g., InceptionV3) with batching. DARIS achieves higher throughput without batching (Section VI).

#### D. How to Predict the Execution Time of DNNs?

Timing prediction is essential for real-time schedulers, but GPU parallelism makes execution times unpredictable. Clockwork [14] ensures predictable WCET by scheduling one DNN at a time, sacrificing throughput for predictability. Multi-tenant systems face resource contention, leading to variability and overly conservative WCET estimates. Wang *et al.* use history-based WCET from 10-minute simulations [15], also yielding conservative results. We propose a dynamic history-based prediction (Figure 9) for more optimistic estimates for soft real-time systems.

### III. SYSTEM MODEL

#### A. Task Model

We consider a set of  $N_c$  contexts, each with  $N_{SM}$  SMs. Periodic tasks correspond to individual DNNs, divided into sub-tasks or stages. We define a task set  $TS = \{\tau_1, \dots, \tau_{N_{ts}}\}$  with  $N_{ts}$  tasks, where each task  $\tau_i$  has  $n_i$  sequential sub-tasks ( $\tau_i = \{\tau_{i,1}, \dots, \tau_{i,n_i}\}$ ). Tasks have two priority levels: high-priority (HP) and low-priority (LP), with counts  $N_h$  and  $N_l$ , respectively so that  $N_{ts} = N_h + N_l$ .

Each task is represented as  $\tau_i(T_i, D_i, mret_i(t), p_i, ctx_i(t))$ , where  $T_i$  is the period,  $D_i$  the relative deadline,  $mret_i(t)$  the Maximum Recent Execution Time (MRET) at time  $t$ ,  $p_i$  the priority level, and  $1 \leq ctx_i(t) \leq N_c$  the current context. Deadlines are set equal to periods ( $D_i = T_i$ ). MRET is used instead of WCET to avoid pessimistic estimates common with colocated DNNs (see Section III-B2). A stage is defined as  $\tau_{i,j}(D_{i,j}(t), mret_{i,j}(t))$ , where  $D_{i,j}$  is the virtual deadline (determined using Equation (8)). Figure 2 shows task stages and their virtual relative deadlines based on MRET.

#### B. Definitions

1) *Staging*: To be able to enforce priorities more efficiently on a smaller scale, we introduce a synchronization-based, coarse-grained preemption mechanism called *staging*. Synchronization points can be placed after a few kernels [21], after each layer [22], after several layers [23], or even dynamically [24]. Excessive synchronization, however, can reduce GPU utilization, as deep learning frameworks release kernels asynchronously to optimize throughput. We segment DNNs into sequential stages/sub-tasks, allowing preemption only at these boundaries. Stages are selected based on the DNN's logical structure; for instance, ResNet [25] is divided into four stages.

2) *Maximum Recent Execution Time (MRET)*: We propose a dynamic WCET estimation, *Maximum Recent Execution Time* (MRET), capturing the maximum execution time within a recent window,

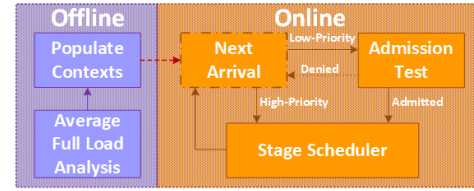


Fig. 3: Proposed Scheduler

adapting to workload changes. MRET is computed per stage rather than for the entire task (Figure 2).

$$mret_{i,j}(t) = \max_{p \in \{t-ws, \dots, t-1\}} et_{i,j}(p) \quad (1)$$

$$mret_i(t) = \sum_{1 \leq j \leq n_i} mret_{i,j}(t), \quad (2)$$

Here,  $ws$  is the window size;  $mret_{i,j}$  and  $et_{i,j}(t)$  are the MRET and actual execution time of  $\tau_{i,j}$  at time  $t$ .

3) *Utilization*: In DARIS, we define the utilization of a task as:

$$u_i(t) = \frac{mret_i(t)}{T_i} \quad (3)$$

We use Equations (4) to (6) to measure the utilization of a context:

$$U_k^{h,t}(t) = \sum_{\substack{1 \leq i \leq N_h \\ ctx_i = k}} u_i^h(t) \quad (4)$$

$$U_k^{l,t}(t) = \sum_{\substack{1 \leq i \leq N_l \\ ctx_i = k}} u_i^l(t) \quad (5)$$

$$U_k^t(t) = U_k^{h,t}(t) + U_k^{l,t}(t) \quad (6)$$

where  $U_k^t(t)$  is the total utilization of context  $k$ , and  $U_k^{h,t}(t)$  and  $U_k^{l,t}(t)$  are the total utilization of HP and LP tasks in context  $k$ , respectively. We will use Equation (6) for load balancing between contexts in the offline phase. For the admission test during the online phase, we will rely on the active utilization:

$$U_k^a(t) = U_k^{h,t}(t) + U_k^{l,a}(t) \quad (7)$$

The active utilization is defined as the sum of the total utilization of the HP tasks ( $U_k^{h,t}(t)$ ) and the total utilization of *currently active* LP tasks ( $U_k^{l,a}(t)$ ), i.e., LP tasks that have an active job released but not yet finished.

4) *Virtual Deadline*: The task deadlines are usually an application requirement set by the designer. We introduce the *virtual deadline*, an intermediary parameter assigned to each stage. The virtual deadline for  $\tau_{i,j}$  is calculated as:

$$D_{i,j}(t) = \frac{mret_{i,j}(t)}{mret_i(t)} D_i \quad (8)$$

Figure 2 illustrates the relationship between MRET and virtual deadlines, with longer stages receiving a larger share of the task's total relative deadline ( $D_i$ ).

#### C. Hyperparameters

1) *Number of Parallel Tasks*: Each stream can execute one task at a time, with the total number of streams determining the maximum number of jobs. We use  $N_c$  contexts, each containing  $N_s$  streams, allowing a maximum of  $N_p = N_c \times N_s$  concurrent DNNs. The hyperparameters  $N_c$  and  $N_s$  define the desired parallel task count and need to be determined. Three policies, based on the context and stream numbers, are discussed in Section V.

TABLE I: Batching performance of different DNNs

| DNN         | min (JPS) | max (JPS) | batching gain |
|-------------|-----------|-----------|---------------|
| ResNet18    | 627       | 1025      | 1.63x         |
| ResNet50    | 250       | 433       | 1.73x         |
| UNet        | 241       | 260       | 1.08x         |
| InceptionV3 | 142       | 446       | 3.13x         |

2) *Oversubscription Level and Number of SMs*: All contexts are allocated an equal number of SMs as:

$$N_{SM} = \text{ceil}_{\text{even}} \left( \frac{OS \times N_{SM, \max}}{N_c} \right) \quad (9)$$

where  $\text{ceil}_{\text{even}}$  rounds up to the nearest even number,  $N_{SM, \max}$  is the total GPU SMs, and  $OS$  is the oversubscription value, constrained to  $1 \leq OS \leq N_c$ .  $OS = 1$  assigns each context its own SMs, with  $OS = N_c$  context will share all SMs, anything in between has a variable level of SM sharing (i.e. lower values reduces interference while higher values enhance utilization through increased sharing).

#### IV. REAL-TIME SCHEDULER

We propose a multi-tenant inference scheduler for real-time DNNs, where each task corresponds to a distinct DNN and is classified as either HP or LP. Our scheduler (Figure 3) has two phases: offline and online. The offline phase sets the initial state, while the online phase is the main body of the scheduler.

##### A. Offline Phase

In the offline phase, we allocate  $N_c$  contexts to tasks. HP tasks are given fixed contexts, while LP tasks can migrate between contexts as needed. This initial assignment prioritizes load balancing across contexts, establishing an efficient starting point for the online phase.

1) *Average Full Load Analysis*: In the offline phase, with no measurement history, MRET cannot be used. Instead, we compute the *Average Full-Load Execution Time* (AFET) by executing the target task in one stream while randomly executing others in the remaining streams, providing a pessimistic initialization metric. AFET is replaced in later iterations as it does not adapt to recent load trends (existing WCET estimates can be used if desired). Substituting  $afet_i$  for  $mret_i(t)$  in Equation (3) when  $t = 0$ , task utilization is computed as:

$$u_i(t) = \begin{cases} \frac{afet_i}{T_i} & \text{if } t = 0 \\ \frac{mret_i(t)}{T_i} & \text{if } t \geq 1 \end{cases} \quad (10)$$

---

##### Algorithm 1 Initial Context Assignment

---

```

1: procedure POPULATECONTEXTS( )
2:   // pool : context pool
3:   for all task in highTasks do
4:     ctx ← minUtil(pool)
5:     task.context ← ctx
6:     ctx.totalUtil ← ctx.totalUtil + task.util
7:   end for
8:   for all task in lowTasks do
9:     ctx ← minUtil(pool)
10:    task.context ← ctx
11:    ctx.totalUtil ← ctx.totalUtil + task.util
12:   end for
13: end procedure

```

---

TABLE II: Task sets

| Name        | #High | #Low | Task JPS |
|-------------|-------|------|----------|
| ResNet18    | 17    | 34   | 30       |
| UNet        | 5     | 10   | 24       |
| InceptionV3 | 9     | 18   | 24       |

2) *Populating Contexts*: The objective is to assign tasks to contexts such that total context utilization ( $U_k^t(0)$ ), HP ( $U_k^{h,t}(0)$ ), and LP ( $U_k^{l,t}(0)$ ) task utilization are balanced across contexts. Algorithm 1 presents the pseudo-code for the initial context assignment. Lines 3 to 7 assigns HP tasks to contexts, while Lines 8 to 12 distributes LP tasks to balance utilization.

##### B. Online Phase

The goal of the online phase is to promptly schedule HP tasks, minimize deadline misses, and maximize throughput. This phase consists of two main components, discussed in the following sections.

1) *Admission Test*: A utilization-based admission mechanism for LP tasks is preferable when demand exceeds capacity. LP tasks undergo a utilization-based test within each context. HP tasks reserve a portion of context utilization, leaving the remaining capacity for LP tasks, which are subject to this utilization test. Given multiple streams ( $N_s$ ) within each context, the utilization test for  $ctx_k$  at time  $t$  is defined as:

$$U_k^r(t) = N_s - U_k^{h,t}(t) \quad (11)$$

$$U_k^{l,a}(t) + u_j(t) < U_k^r(t) \quad (12)$$

Here,  $U_k^r(t)$  is the remaining utilization of context  $k$ . If a job satisfies Equation (12) for  $k = ctx_i(t)$ , it will be scheduled in context  $k$ . Otherwise, we will test other contexts ( $k \in \{1, \dots, N_c\} \setminus \{ctx_i(t)\}$ ) as migrations candidates. If any of them satisfy Equation (12),  $\tau_i$  will be migrated to the context with earliest predicted finish time. If no context is found that passes the admission test, the task is rejected.

2) *Stage Scheduler*: We extend task priorities from two to eight fixed levels for stages. Stages from HP tasks always take precedence over LP tasks, with the last stage of each task ( $\tau_{i,n_i}$ ) prioritized higher. Any stage with an immediate preceding missed virtual deadline has the next priority level (e.g., if  $\tau_{i,j}$  misses its deadline,  $\tau_{i,j+1}$  receives higher priority). This hierarchy ensures HP tasks are serviced first, emphasizes the final stage of each task to prevent overall deadline misses, and prevents cascading misses by prioritizing stages with preceding deadline misses. Finally, EDF within fixed priority levels ensures tasks with approaching deadlines are prioritized.

#### V. IMPLEMENTATION AND SETUP

We evaluated DARIS on an RTX 2080 Ti GPU implemented using LibTorch (PyTorch's C++ API). Backend modifications enabled multi-context support via thread-local variables. PyTorch's multi-device API was customized to treat each device as a context while replacing device-level with context-level synchronizations to avoid inter-context locks. Key components for streams, memory, cache, and cuBLAS were updated to support our algorithm.

Our benchmarks include three DNNs: *ResNet* [25], *UNet* [26], and *InceptionV3* [27], each with 224x224x3 input. Throughput gains from batching, evaluated as an upper limit for throughput without colocation, are reported in Figure 1 and Table I (throughput in *Jobs Per Second* (JPS)). ResNet, with a linear design, is widely used, while UNet's wide architecture and skip connections make it memory-heavy, achieving only a 1.08x from batching. InceptionV3, with multiple parallel paths, exhibits the highest batching gain of 3.13x, highlighting its reliance on batching.

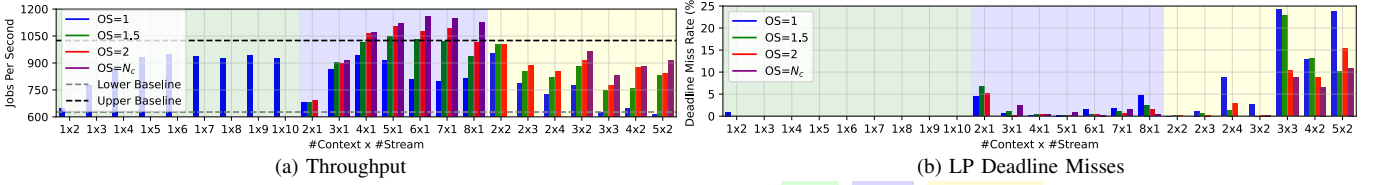


Fig. 4: Scheduling results for ResNet18 task set (STR, MPS, MPS+STR)

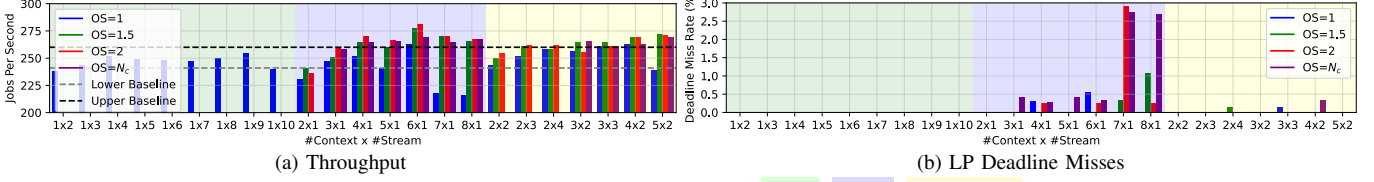


Fig. 5: Scheduling results for UNet task set (STR, MPS, MPS+STR)

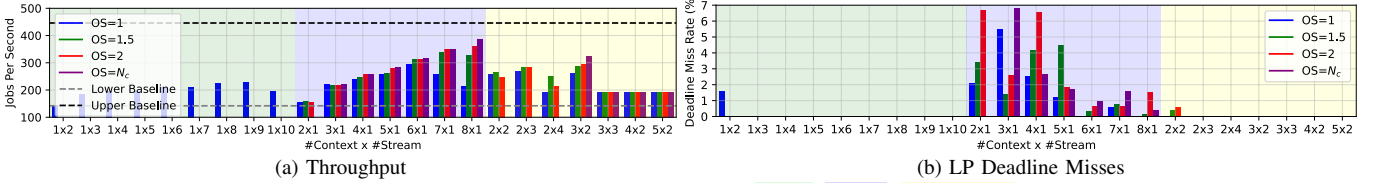


Fig. 6: Scheduling results for InceptionV3 (STR, MPS, MPS+STR)

We use MPS and CUDA Streams with oversubscription to colocate DNNs, aiming to minimize deadline misses and maximize throughput. DARIS variations are assessed using three partitioning policies:

- 1) **STR**: Exclusively uses streams for scheduling DNNs.
- 2) **MPS**: Relies solely on MPS for scheduling DNNs.
- 3) **MPS+STR**: Combines MPS and streams to assess their joint effect on timeliness and throughput.

We schedule 2–10 parallel DNNs ( $2 \leq N_p \leq 10$ ). Four oversubscription options are explored:  $OS \in \{1, 1.5, 2, N_c\}$ , where  $OS = 1$  indicates no SM sharing and  $OS = N_c$  represents full SM sharing.

Performance is assessed using total JPS and Deadline Miss Rate (DMR) for throughput and timeliness, with configurations denoted as  $N_c \times N_s$  or  $N_c \times N_s \text{--} OS$ . Three main task sets (Table II), each tied to a DNN type, are tested with 30 jobs/sec for ResNet18 and 24 jobs/sec for others. Experiments are run under 150% overload, using the upper baseline as full load due to varying maximum loads across different configurations. A 2:1 LP-to-HP task ratio is maintained, with alternative ratios explored in Section VI-I.

## VI. EXPERIMENTAL RESULTS

Main scenario results are presented in Figures 4 to 6. Throughput figures (Figures 4a to 6a) compare with lower (single DNN) and upper (pure batching) baseline throughput from Table I. Although meeting every deadline is not guaranteed, we did not observe any HP deadline misses, and only LP DMRs are shown in Figures 4b to 6b. DMR is the ratio of missed deadlines to accepted jobs.

1) **Throughput**: As shown in Figures 4a to 6a, MPS delivers the highest throughput across all DNNs. ResNet18 and UNet achieve peak throughput at  $N_c = 6$ , while InceptionV3 benefits from increased concurrency up to  $N_c = 8$ . The MPS+STR policy outperforms STR, showing that multiple contexts enhance throughput on MPS-enabled GPUs. For ResNet18 and UNet, we exceeded the pure batching baseline (Table I) by 13% (1158 JPS vs. 1025 JPS) and 8% (281 JPS vs. 260 JPS) respectively, without batching. However, for InceptionV3, we only achieved 87% of its upper baseline. Its complex, narrow architecture limits throughput. We conducted an experience to release parallel paths of InceptionV3 using distinct

streams to improve throughput but gained only 9% in throughput still below the upper baseline. By using batching instead, we were able to surpass upper baseline for InceptionV3 which will be discussed in Section VI-H.

### A. Deadline Misses

DARIS prioritizes minimizing HP task response time to reduce the chance of deadline misses, though it does not guarantee meeting every deadline. In our main experiments, no HP deadline misses were observed. From here on, DMR refers to LP DMR unless stated otherwise. Except for less than 2% DMR in the  $1 \times 2$  configuration for InceptionV3, no deadline misses occurred with the STR policy. Each context uses a dedicated job queue, leaving STR with a single global queue. While this may reduce throughput, it ensures optimal timeliness. UNet showed the lowest DMR, peaking at less than 3%, with only 0.25% at its best throughput configuration ( $6 \times 1_2$ ). The MPS+STR policy produced the worst DMR, reaching 25% for ResNet18. Meanwhile, with the MPS policy, both ResNet18 and InceptionV3 maintained DMRs below 7%, with around 2% at their peak throughput configurations ( $6 \times 1_6$  and  $8 \times 1_8$ , respectively).

### B. Comparison with State-of-the-Art

GSlice [10] is a state-of-the-art inference server, achieving 1152 JPS for ResNet50 with batching and 1193 JPS with GSlice—a 3.5% gain. In comparison, we achieved 433 JPS with batching and 498 JPS using DARIS on our hardware, improving throughput by 15% over batching and 11.5% over GSlice. Without oversubscription, DARIS throughput drops to 374 JPS, 8% below batching. Inference servers like [10], [14] generally allow some deadline misses without detailed rates. The closest comparison, [15], reports no HP deadline misses and up to 12% for LP tasks. In our case, LP deadline misses are below 2% in the best and under 7% in the worst scenario with the MPS policy, and zero with the STR policy. RTGPU [28] lacks task prioritization, reporting up to 11% overall deadline misses.

### C. DARIS Policy Comparison

Overall observations indicate that the MPS policy offers the best throughput, while STR results in ideal DMR. Although MPS may not



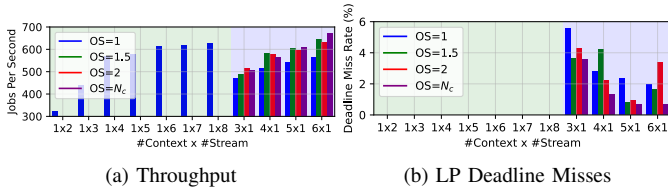


Fig. 7: Scheduling results for mixed task set (STR, MPS)

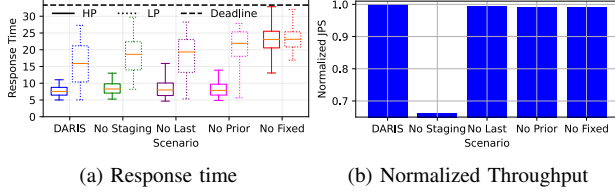


Fig. 8: Response time and throughput for different scenarios

achieve the lowest DMR, it remains below 7% in all configurations. For many applications, this DMR is acceptable, but for those with stringent constraints, *STR* is the safest policy. In scenarios with embedded GPUs lacking MPS support, *STR* is the sole feasible option. Conversely, the *MPS+STR* policy yields the least favorable outcomes, with subpar throughput and DMR across all configurations. Also, UNet has the most consistent performance in all scenarios, making it the least sensitive DNN to concurrency mechanisms and configurations.

#### D. Mixed Task Set

It is essential to analyze a mixed task set with all DNN types, reflecting real-world scenarios. The results for the mixed task set are shown in Figure 7. As with individual task sets, the *MPS* policy achieves the highest throughput, while the *STR* policy offers the most reliable deadline performance.

#### E. Oversubscribing SMs

Contrary to some claims in the literature [18]–[20], our study shows that, in real-time scenarios without batching, oversubscription significantly boosts throughput. While its impact on timeliness is not strictly monotonic, higher *OS* values generally result in fewer deadline misses. Isolating SMs ( $OS = 1$ ) leads to a sharp drop in throughput. Our findings confirm that oversubscription consistently benefits both throughput and timeliness. For wide DNNs like UNet, which gain little from batching, 200% SM oversubscription is sufficient, while narrower DNNs require even higher oversubscription. Although more concurrent DNNs combined with oversubscription can increase resource contention, it also enhances throughput. The goal is to identify a trade-off point to maximize benefit.

#### F. DARIS Modules Contribution

Works addressing real-time constraints often treat meeting deadlines or minimizing latency as binary outcomes [10], [14], [15], [28]. However, enhancing the quality of service by minimizing response time is crucial for better responsiveness. We conducted experiments to evaluate how different DARIS modules impact overall performance. Figure 8a shows the response times, and Figure 8b presents normalized throughput for DARIS and four alternative scenarios using ResNet18:

- *No Staging*: tasks treated as whole units without staging
- *No Last*: last stages of tasks are not prioritized
- *No Prior*: no high priority when preceding stage misses deadline
- *No Fixed*: no differentiation in task priority among stages

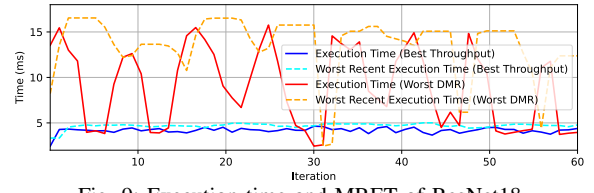


Fig. 9: Execution time and MRET of ResNet18

The original DARIS achieves response times of 5–12 ms for HP tasks and 5–27.5 ms for LP tasks, making HP tasks finish roughly 2.5 times faster. In *No Staging* scenario, throughput drops by 33%, and response times increase due to the lack of preemption, with 5.5% and 22.5% deadline misses for HP and LP tasks, respectively. *No Last* scenario increases worst-case response times for HP tasks by 38% without significantly affecting throughput. In *No Prior* scenario, average response times rise for all tasks. *No Fixed* scenario, which removes inter-task priority differentiation, results in a 2.5% deadline miss rate for both task priorities. These findings highlight the effectiveness of DARIS modules, emphasizing the importance of *staging* and *task priority* in improving throughput and meeting deadlines.

#### G. Maximum Recent Execution Time

We introduced MRET to address the unpredictability of GPU execution time and avoid the pessimism of history-based WCET approaches. Figure 9 shows the actual execution time and MRET for ResNet18 under the best throughput configuration ( $6 \times 1_6$ ) and the worst deadline miss rate configuration ( $3 \times 3_1$ ). We selected a window size ( $ws$ ) of 5, as smaller values increase DMR, while larger values reduce throughput. With the  $6 \times 1_6$  configuration, MRET accurately predicts execution time in most cases, whereas in the  $3 \times 3_1$  configuration, execution time often exceeds MRET predictions.

#### H. Batching

We conducted an additional experiment to demonstrate the impact of batching combined with DARIS, using batch sizes of 4, 2, and 8 for ResNet18, UNet, and InceptionV3, respectively. Figures 10a to 10c present the throughput results. Our key observation is that fewer parallel tasks are needed to exceed the upper baseline, with decent throughput achieved even with 1 or 2 parallel tasks. We also observed similar benefits from SM oversubscription compared to the main experiment (Figures 4 to 6) without batching, though the difference is less pronounced.

Figures 10d to 10f show the throughput improvements compared to similar configurations from the main experiment (Figures 4a to 6a). UNet exhibits the smallest gain, with improvements up to 18%, reaffirming its efficient architecture in utilizing GPU resources even without batching. In contrast, InceptionV3 achieves the highest improvement, with at least a 55% increase over the main experiment without batching. For DMR (Figures 10g to 10i), ResNet18 and InceptionV3 show slightly better DMRs compared to the main experiment (Figure 4b and Figure 6b). UNet demonstrates the most significant improvement, with its DMR reduced to under 0.5%.

#### I. Overloading And Task Ratio

We conducted a final experiment to examine the scheduler’s behavior under different HP-to-LP task ratios. Figure 11 presents the results for throughput and deadline miss rates for both priorities, using ResNet18 and UNet in full load and overloaded scenarios. We also evaluate an overloaded scenario where HP tasks must pass the admission test (*Overload+HPA*).

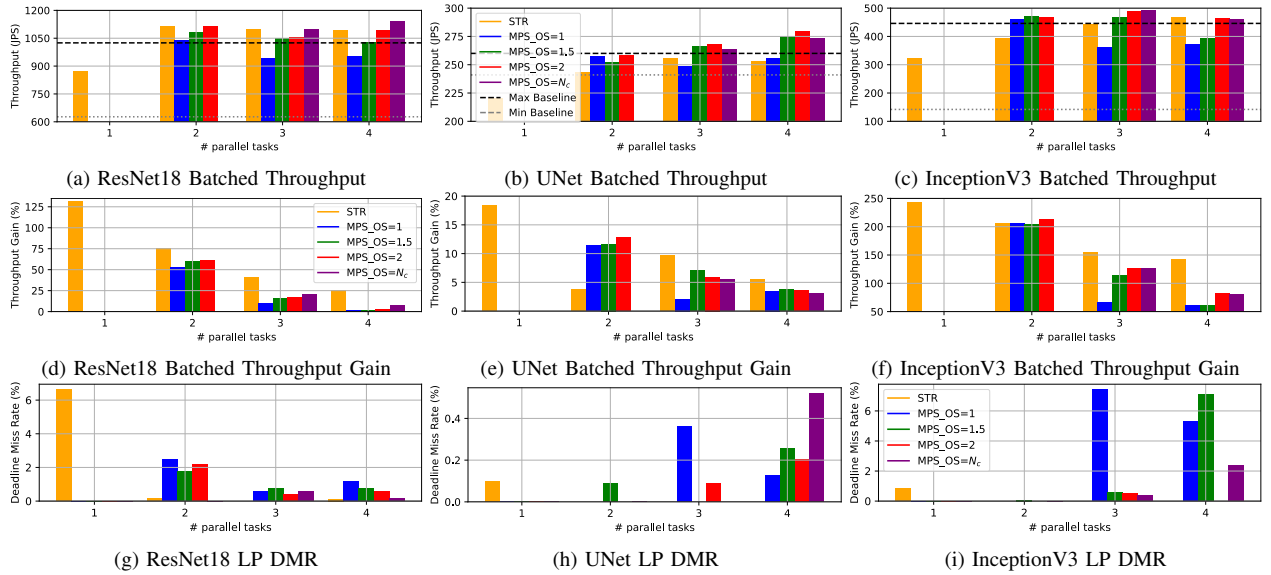


Fig. 10: Absolute Throughput When Batching

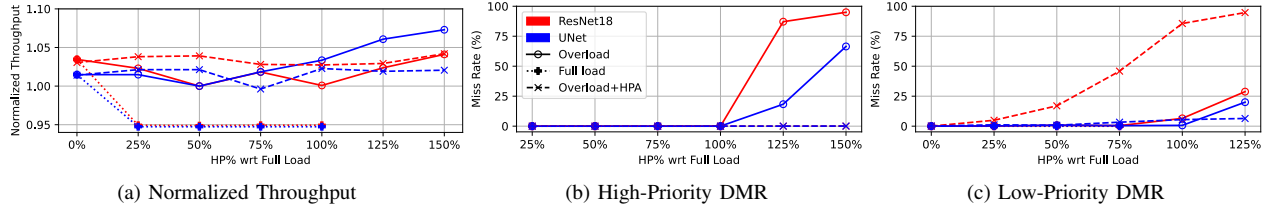


Fig. 11: Overloading with different HP to LP ratios

As shown in Figure 11a, throughput remains stable across different task ratios and overload scenarios. In the full load scenario, throughput drops consistently by 5% with the presence of LP tasks, but no HP or LP deadline misses occur. However, in the overload scenario, when HP load exceeds 100% of full capacity, DMR for HP tasks rises sharply since HP tasks are scheduled without an admission test. Although this ensures all HP tasks are scheduled, it results in exponentially increasing deadline misses when the system is overwhelmed by HP tasks.

To address this, we introduce *Overload+HPA*, ensuring zero deadline misses for HP tasks by applying the admission test, even when the system is overloaded with HP tasks. The trade-off is that some HP tasks may be dropped, and DMR for LP tasks increases. However, UNet avoids this downside due to its wide efficient architecture. We recommend limiting HP tasks to 50% of the full load to minimize deadline misses, as shown in the main experiments. In cases of overloaded HP tasks, using the *Overload+HPA* approach ensures safer scheduling with minimal HP deadline misses.

## VII. CONCLUSIONS

In this work, we presented DARIS, a novel real-time GPU scheduler designed for periodic tasks in a multi-tenant DNN setup with task priorities. DARIS applies oversubscribed spatio-temporal scheduling to minimize deadline misses for high-priority tasks while maximizing throughput by scheduling as many low-priority tasks as possible. For spatial partitioning, we used both MPS and CUDA streams to colocate multiple DNNs. Through extensive experiments, we compared the throughput and timeliness achieved by using MPS and streaming separately and in combination. We found that

MPS provides the highest throughput, while streaming yields the lowest deadline miss rate. Additionally, we evaluated DARIS under batched inputs, overloaded scenarios, and various task ratios. Our experiments showed that oversubscribing SMs can boost throughput significantly, even in the presence of batched inputs and DNNs with wide architecture.

For temporal scheduling, we used staging as synchronization points to enable coarse-grained preemption, improving throughput and reducing deadline misses. By prioritizing stages based on task priority, stage order, and past missed deadlines, we further reduced deadline misses and enhanced response times to improve the quality of service. While we explored leveraging the parallel paths of non-linear DNNs to boost throughput, we found that batched inputs provided superior results in managing parallel paths.

## ACKNOWLEDGMENT

This work was supported in part by NSF under grant number CPS-2038726 and by the Commonwealth Cyber Initiative, an investment in the advancement of cyber R&D, innovation, and workforce development. For more information about CCI, visit [www.cyberinitiative.org](http://www.cyberinitiative.org).

## REFERENCES

- [1] X. He, J. Liu, Z. Xie, *et al.*, “Enabling energy-efficient dnn training on hybrid gpu-fpga accelerators,” in *Proceedings of the ACM International Conference on Supercomputing*, 2021, pp. 227–241.
- [2] W. Xu, Y. Zhang, and X. Tang, “Parallelizing dnn training on gpus: Challenges and opportunities,” in *Companion Proceedings of the Web Conference 2021*, 2021, pp. 174–178.
- [3] Z. Ye, W. Gao, Q. Hu, *et al.*, “Deep learning workload scheduling in gpu datacenters: A survey,” *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–38, 2024.

- [4] L. Liu, Z. Dong, Y. Wang, and W. Shi, "Prophet: Realizing a predictable real-time perception pipeline for autonomous vehicles," in *2022 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2022, pp. 305–317.
- [5] S. Roshan, J. Tanha, M. Zarrin, A. F. Babaei, H. Nikkhah, and Z. Jafari, "A deep ensemble medical image segmentation with novel sampling method and loss function," *Computers in Biology and Medicine*, p. 108305, 2024.
- [6] S. H. Mostafaei, J. Tanha, A. Sharafkhaneh, Z. H. Mostafaei, M. H. A. Al-Jaf, and A. F. Babaei, "An ensemble model for sleep stages classification," in *2023 31st International Conference on Electrical Engineering (ICEE)*, IEEE, 2023, pp. 327–332.
- [7] Z. Aghapour, S. Sharifian, and H. Taheri, "Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed ai execution tasks in iot edge computing environments," *Computer Networks*, vol. 223, p. 109577, 2023.
- [8] Y. Li, Z. Li, W. Yang, and C. Liu, "Rt-lm: Uncertainty-aware resource management for real-time inference of language models," *arXiv preprint arXiv:2309.06619*, 2023.
- [9] A. F. Babaei, J. Tanha, A. F. Babaei, and M. G. Matin, "A hybrid deep learning model for 5-digit handwritten recognition," in *2024 10th International Conference on Artificial Intelligence and Robotics (QICAR)*, IEEE, 2024, pp. 329–333.
- [10] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan, "Gslice: Controlled spatial sharing of gpus for a scalable inference platform," in *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 492–506.
- [11] K. Guo, Y. Xu, Z. Qi, and H. Guan, "Optimum: Runtime optimization for multiple mixed model deployment deep learning inference," *Journal of Systems Architecture*, vol. 141, p. 102901, 2023.
- [12] F. Yu, D. Wang, L. Shangguan, M. Zhang, C. Liu, and X. Chen, "A survey of multi-tenant deep learning inference on gpu," *arXiv preprint arXiv:2203.09040*, 2022.
- [13] W. Zhang, W. Cui, K. Fu, *et al.*, "Laius: Towards latency awareness and improved utilization of spatial multitasking accelerators in datacenters," in *Proceedings of the ACM international conference on supercomputing*, 2019, pp. 58–68.
- [14] A. Gujarati, R. Karimi, S. Alzayat, *et al.*, "Serving {dnns} like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 443–462.
- [15] Y. Wang, M. Karimi, Y. Xiang, and H. Kim, "Balancing energy efficiency and real-time performance in gpu scheduling," in *2021 IEEE Real-Time Systems Symposium (RTSS)*, IEEE, 2021, pp. 110–122.
- [16] A. F. Babaei and T. Chantem, "Sgprs: Seamless gpu partitioning real-time scheduler for periodic deep learning workloads," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, IEEE, 2024, pp. 1–2.
- [17] NVIDIA Corporation, *NVIDIA Multi-Process Service Documentation*, <https://docs.nvidia.com/deploy/mps/index.html>.
- [18] A. Dhakal, K. Ramakrishnan, S. G. Kulkarni, P. Sharma, and J. Cho, "Slice-tune: A system for high performance dnn autotuning," in *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, 2022, pp. 228–240.
- [19] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan, "D-stack: High throughput dnn inference by effective multiplexing and spatio-temporal scheduling of gpus," *arXiv preprint arXiv:2304.13541*, 2023.
- [20] G. Rattihalli, N. Hogade, A. Dhakal, *et al.*, "Fine-grained heterogeneous execution framework with energy aware scheduling," in *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, IEEE, 2023, pp. 35–44.
- [21] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent {gpu-accelerated}-{dnn} inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 539–558.
- [22] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, "{Pipeswitch}: Fast pipelined context switching for deep learning applications," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 499–514.
- [23] H. Zhang, Y. Tang, A. Khandelwal, and I. Stoica, "{Shepherd}: Serving {dnns} in the wild," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 787–808.
- [24] Z. Xia, Y. Hao, J. Duan, C. Wang, and J. Jiang, "Towards optimal preemptive gpu time-sharing for edge model serving," in *Proceedings of the 9th International Workshop on Container Technologies and Container Clouds*, 2023, pp. 13–18.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [26] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, Springer, 2015, pp. 234–241.
- [27] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [28] A. Zou, J. Li, C. D. Gill, and X. Zhang, "Rtgpu: Real-time gpu scheduling of hard deadline parallel tasks with fine-grain utilization," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1450–1465, 2023.